

Домашнее задание: метод внутренней точки (следование по центральному пути и метод Мехротры)

Введение

Цель задания — реализовать прямо–двойственный метод внутренней точки для задачи линейного программирования в канонической форме

$$\max c^\top x \quad \text{при} \quad Ax \leq b, \quad x \geq 0,$$

где $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$.

Нужно реализовать **один** из двух вариантов:

- **Вариант 1 (базовый):** infeasible path-following (демпфированный ньютоновский шаг по центральному пути).
- **Вариант 2 (повышенная сложность):** Mehrotra predictor–corrector (аффинный предиктор + корректирующий шаг).

Оценивание: за корректную разреженную линейную алгебру (ККТ/нормальные уравнения, переиспользование факторизации, сторонние sparse-библиотеки) выставляется более высокий балл.

0. Ввод/вывод

0.1 Формат входного файла

Входной текстовый файл задаёт A, b, c :

```
n m
c1 c2 ... cn
a11 a12 ... a1n b1
a21 a22 ... a2n b2
...
am1 am2 ... amn bm
```

Первая строка: n (переменные), m (ограничения). Далее c и m строк $(A \mid b)$.

0.2 Требуемый формат вывода

Требуемые выходные данные (и только они):

- **status** (строка: `optimal` / `infeasible` / `unbounded` / `max_iter`);
- **x_star** (вектор x^* длины n , выводить **только** если `status=optimal`).

1. Slack-переменные и двойственная задача

Переходим от $Ax \leq b$ к равенствам, вводя slack-переменные $w \geq 0$:

$$Ax + w = b, \quad x \geq 0, \quad w \geq 0.$$

Двойственная задача к $\max c^\top x$ при $Ax \leq b, \quad x \geq 0$:

$$\min b^\top y \quad \text{при} \quad A^\top y - z = c, \quad y \geq 0, \quad z \geq 0.$$

Метод внутренней точки поддерживает строго положительную точку:

$$x > 0, \quad w > 0, \quad y > 0, \quad z > 0.$$

2. Центральный путь (4 уравнения)

Определим диагональные матрицы

$$X = \text{diag}(x), \quad Z = \text{diag}(z), \quad Y = \text{diag}(y), \quad W = \text{diag}(w),$$

и вектор единиц $e = (1, 1, \dots, 1)^\top$ подходящей размерности.

Центральный путь задаётся системой:

$$Ax + w = b, \tag{1}$$

$$A^\top y - z = c, \tag{2}$$

$$XZ e = \mu e, \tag{3}$$

$$YW e = \mu e, \tag{4}$$

где $\mu > 0$ стремится к нулю.

2.1 Комплементарный зазор (скаляр)

Определим duality gap (комплементарный зазор)

$$\gamma := x^\top z + y^\top w, \quad \bar{\mu} := \frac{\gamma}{n + m}.$$

Важно: γ — **скаляр**. Векторные правые части комплементарности задаются отдельно.

3. Ньютоновский шаг и линейные системы

3.1 Невязки и правые части

Невязки допустимости:

$$\rho := b - Ax - w \in \mathbb{R}^m, \quad \sigma := c - A^\top y + z \in \mathbb{R}^n.$$

Для выбранного целевого μ зададим векторные правые части комплементарности:

$$r_{xz} := \mu e - XZ e \in \mathbb{R}^n, \quad r_{yw} := \mu e - YW e \in \mathbb{R}^m.$$

3.2 Несимметричная ньютоновская система (прямая линеаризация)

Линеаризация (1)–(4) даёт:

$$A\Delta x + \Delta w = \rho, \quad (5)$$

$$A^\top \Delta y - \Delta z = \sigma, \quad (6)$$

$$Z\Delta x + X\Delta z = r_{xz}, \quad (7)$$

$$W\Delta y + Y\Delta w = r_{yw}. \quad (8)$$

3.3 Симметричная ККТ-система (размер $2n + 2m$)

Для устойчивого решения обычно используют симметричную форму в порядке неизвестных $(\Delta z, \Delta y, \Delta x, \Delta w)$:

$$\begin{bmatrix} -XZ^{-1} & 0 & -I & 0 \\ 0 & 0 & A & I \\ -I & A^\top & 0 & 0 \\ 0 & I & 0 & YW^{-1} \end{bmatrix} \begin{bmatrix} \Delta z \\ \Delta y \\ \Delta x \\ \Delta w \end{bmatrix} = \begin{bmatrix} -Z^{-1}r_{xz} \\ \rho \\ \sigma \\ W^{-1}r_{yw} \end{bmatrix}. \quad (9)$$

Эквивалентная запись RHS, часто встречающаяся в литературе: $-Z^{-1}r_{xz} = -\mu Z^{-1}e + x$, $W^{-1}r_{yw} = \mu W^{-1}e - y$.

3.4 Редуцированная симметричная система (размер $n + m$)

Исключим Δz и Δw из (5)–(8):

$$\Delta z = A^\top \Delta y - \sigma, \quad \Delta w = \rho - A\Delta x.$$

Тогда система на $(\Delta x, \Delta y)$ имеет вид:

$$\begin{bmatrix} X^{-1}Z & A^\top \\ A & -Y^{-1}W \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} \sigma + X^{-1}r_{xz} \\ \rho - Y^{-1}r_{yw} \end{bmatrix}. \quad (10)$$

После нахождения $(\Delta x, \Delta y)$ восстановите $\Delta z = A^\top \Delta y - \sigma$ и $\Delta w = \rho - A\Delta x$.

3.5 Нормальные уравнения (размер m , SPD)

Из (10) можно исключить Δx и получить нормальные уравнения на Δy :

$$(AXZ^{-1}A^\top + Y^{-1}W) \Delta y = -\rho + Y^{-1}r_{yw} + AXZ^{-1}(\sigma + X^{-1}r_{xz}). \quad (11)$$

Далее восстанавливаем:

$$\begin{aligned} \Delta x &= XZ^{-1}(\sigma + X^{-1}r_{xz} - A^\top \Delta y), \\ \Delta w &= \rho - A\Delta x, \\ \Delta z &= A^\top \Delta y - \sigma. \end{aligned}$$

4. Алгоритм 1: infeasible path-following (базовый)

4.1 Параметры

Рекомендуемые параметры:

$$\varepsilon_p = \varepsilon_d = \varepsilon_\mu = 10^{-8}, \quad \alpha_0 = 0.99, \quad \delta = 0.1, \quad \text{max_iter} = 200.$$

На каждой итерации целевой барьер:

$$\mu := \delta \bar{\mu}.$$

4.2 Инициализация

Допустимо взять простую строго положительную инициализацию:

$$x = e, \quad w = e, \quad y = e, \quad z = e.$$

(Начальная точка не обязана удовлетворять (1)–(2).)

4.3 Итерационный процесс

Для $k = 0, 1, 2, \dots$:

Шаг 1: Вычислить $\rho = b - Ax - w$, $\sigma = c - A^\top y + z$.

Шаг 2: Вычислить $\gamma = x^\top z + y^\top w$, $\bar{\mu} = \gamma / (n + m)$.

Шаг 3: Если

$$\|\rho\|_\infty \leq \varepsilon_p(1 + \|b\|_\infty), \quad \|\sigma\|_\infty \leq \varepsilon_d(1 + \|c\|_\infty), \quad \bar{\mu} \leq \varepsilon_\mu,$$

то вернуть `status=optimal` и $x^* = x$.

Шаг 4: Положить $\mu = \delta \bar{\mu}$; сформировать $r_{xz} = \mu e - XZe$, $r_{yw} = \mu e - YWe$.

Шаг 5: Решить одну из систем: (9), (10) или (11).

Шаг 6: Найти максимально допустимый шаг, сохраняющий положительность всех компонент:

$$\theta_{\max} = \min \left(1, \min_{i: \Delta x_i < 0} \frac{-x_i}{\Delta x_i}, \min_{i: \Delta w_i < 0} \frac{-w_i}{\Delta w_i}, \min_{i: \Delta y_i < 0} \frac{-y_i}{\Delta y_i}, \min_{i: \Delta z_i < 0} \frac{-z_i}{\Delta z_i} \right), \quad \theta = \alpha_0 \theta_{\max}.$$

Шаг 7: Обновить:

$$x \leftarrow x + \theta \Delta x, \quad w \leftarrow w + \theta \Delta w, \quad y \leftarrow y + \theta \Delta y, \quad z \leftarrow z + \theta \Delta z.$$

Если $k = \text{max_iter}$ без выполнения критерия остановки, вернуть `status=max_iter`.

5. Алгоритм 2: Mehrotra predictor–corrector (повышенная сложность)

Вариант 2 требует **двух** решённых систем на итерацию с **одной и той же** матрицей (разные RHS). Подробности и мотивация: [3, 2, 4].

5.1 Шаг предиктора (affine)

Вычислить $\rho, \sigma, \gamma, \bar{\mu}$ как в Алгоритме 1. Затем положить

$$r_{xz}^{\text{aff}} := -XZe, \quad r_{yw}^{\text{aff}} := -YWe.$$

Решить (например) (9) с RHS, построенной из $(\rho, \sigma, r_{xz}^{\text{aff}}, r_{yw}^{\text{aff}})$, и получить аффинные направления $(\Delta x^{\text{aff}}, \Delta w^{\text{aff}}, \Delta y^{\text{aff}}, \Delta z^{\text{aff}})$.

5.2 Прогноз и параметр центрирования

Найти аффинный шаг до границы (можно использовать $\alpha_0 = 1$ для прогноза):

$$\theta_{\max}^{\text{aff}} = \min \left(1, \min_{i: \Delta x_i^{\text{aff}} < 0} \frac{-x_i}{\Delta x_i^{\text{aff}}}, \min_{i: \Delta w_i^{\text{aff}} < 0} \frac{-w_i}{\Delta w_i^{\text{aff}}}, \min_{i: \Delta y_i^{\text{aff}} < 0} \frac{-y_i}{\Delta y_i^{\text{aff}}}, \min_{i: \Delta z_i^{\text{aff}} < 0} \frac{-z_i}{\Delta z_i^{\text{aff}}} \right).$$

Построить прогнозную $\bar{\mu}_{\text{aff}}$:

$$\bar{\mu}_{\text{aff}} = \frac{(x + \theta_{\max}^{\text{aff}} \Delta x^{\text{aff}})^\top (z + \theta_{\max}^{\text{aff}} \Delta z^{\text{aff}}) + (y + \theta_{\max}^{\text{aff}} \Delta y^{\text{aff}})^\top (w + \theta_{\max}^{\text{aff}} \Delta w^{\text{aff}})}{n + m}.$$

Задать (типично степень $p = 3$):

$$\sigma_{\text{cen}} := \left(\frac{\bar{\mu}_{\text{aff}}}{\bar{\mu}} \right)^p \in [0, 1], \quad \mu_{\text{tar}} := \sigma_{\text{cen}} \bar{\mu}.$$

5.3 Шаг корректора (total direction во втором решении)

Сформировать RHS для **полного** шага, включая член второго порядка:

$$r_{xz}^{\text{tot}} := \mu_{\text{tar}} e - XZe - (\Delta x^{\text{aff}} \circ \Delta z^{\text{aff}}), \quad r_{yw}^{\text{tot}} := \mu_{\text{tar}} e - YWe - (\Delta y^{\text{aff}} \circ \Delta w^{\text{aff}}),$$

где $\circ$ — поэлементное произведение.

Важно: во втором решении вы находите *сразу* итоговое направление $(\Delta x, \Delta w, \Delta y, \Delta z)$ (а не “поправку к предиктору”). То есть вы **второй раз** решаете ту же систему (например (9)) с RHS, построенной из $(\rho, \sigma, r_{xz}^{\text{tot}}, r_{yw}^{\text{tot}})$, используя **ту же факторизацию** матрицы.

5.4 Шаг и обновление

Вычислить θ по правилу Алгоритма 1 (с $\alpha_0 \approx 0.99$) и обновить (x, w, y, z) . Критерий остановки — как в Алгоритме 1.

6. Линейная алгебра: про symbolic/numeric факторизацию и оценивание

6.1 Почему это не требуется в базовой реализации

В промышленной sparse-реализации полезно разделять:

- **symbolic analysis** (перестановка, elimination tree, структура fill-in),

- **numeric factorization** (численные значения разложения),

и переиспользовать анализ, так как по итерациям меняются в основном диагональные блоки (X, Z, Y, W).

В **SciPy** доступны разреженные прямые решатели (например, на базе SuperLU), однако типичный пользовательский интерфейс ориентирован на режим “собрать матрицу $\rightarrow$ факторизовать $\rightarrow$ решить” и не предоставляет удобного учебного API уровня “AMD + symbolic analysis один раз $\rightarrow$ затем много раз numeric factorization для матриц с тем же sparsity pattern”. Поэтому в обязательной части допускается решать систему заново на каждой итерации. Подробная индустриальная схема с AMD и разделением Symbolic/Numeric приведена в Приложении А.

6.2 Бонус за сторонние sparse-библиотеки и переиспользование

За более высокий балл приветствуется (см. Приложение А):

- решение (11) разреженным Холецким (SPD);
- использование сторонних библиотек, где есть явный “symbolic + numeric” (например CHOLMOD через `scikit-sparse`);
- в варианте Мехротры — **обязательное** переиспользование одной факторизации для двух RHS (предиктор/корректор).

7. Пример входного файла

Пример ЗЛП (3 переменных, 4 ограничения):

$$\max 3x_1 + 2x_2 + x_3$$

при

$$\begin{aligned} x_1 + x_2 + x_3 &\leq 30, \\ 2x_1 + x_2 &\leq 40, \\ x_1 + 3x_2 + x_3 &\leq 60, \\ x_3 &\leq 10, \\ x &\geq 0. \end{aligned}$$

Файл `example1.txt`:

```
3 4
3 2 1
1 1 1 30
2 1 0 40
1 3 1 60
0 0 1 10
```

8. Что сдавать

1. Исходный код (Python или MATLAB/Octave).

2. README: выбранный вариант (1 или 2), какая система решалась ((9)/(10)/(11)), параметры, критерии остановки.
3. Лог запуска на `example1.txt`: `status` и (если `optimal`) `x_star`.

Список литературы

- [1] R. J. Vanderbei. *Linear Programming: Foundations and Extensions*. Springer, 4th ed., 2014.
- [2] S. J. Wright. *Primal-Dual Interior-Point Methods*. SIAM, 1997.
- [3] S. Mehrotra. On the implementation of a primal-dual interior point method. *SIAM Journal on Optimization*, 2(4):575–601, 1992.
- [4] J. Gondzio. Interior point methods 25 years later. *European Journal of Operational Research*, 218(3):587–601, 2012.

А AMD-переупорядочение и разделение Symbolic/Numeric факторизации

А.1 Зачем это нужно в IPM

В методе внутренней точки матрица ограничений A фиксирована, а на итерациях меняются только диагональные матрицы

$$X = \text{diag}(x), \quad Z = \text{diag}(z), \quad Y = \text{diag}(y), \quad W = \text{diag}(w),$$

то есть меняются *численные значения*, но (в типичном случае) не меняется *шаблон разреженности* (positions of non-zeros) матриц, которые нужно факторизовать.

Наиболее удобные для разреженной линейной алгебры формы систем в IPM:

- **Нормальные уравнения (SPD)** на Δy :

$$M_k \Delta y = \text{rhs}_k, \quad M_k := AX_k Z_k^{-1} A^\top + Y_k^{-1} W_k,$$

- **Редуцированная симметричная форма (quasi-definite)** на $(\Delta x, \Delta y)$:

$$K_k \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = r_k, \quad K_k := \begin{bmatrix} X_k^{-1} Z_k & A^\top \\ A & -Y_k^{-1} W_k \end{bmatrix}.$$

В обеих формах A определяет основной pattern, а блоки, зависящие от x, w, y, z , являются диагональными. Это означает, что можно отделить:

- **Symbolic analysis** (один раз): построение перестановки (например, AMD), дерева исключения, структуры fill-in и выделение памяти;
- **Numeric factorization** (много раз): пересчёт численных значений факторов для новой матрицы с тем же pattern;
- **Solve** (много раз): прямой/обратный ход для разных RHS.

Такой режим является стандартом для эффективных реализаций IPM на разреженных задачах.

А.2 Что делает AMD и как применяется перестановка

AMD (Approximate Minimum Degree) строит перестановку p по **графу разреженности** матрицы (по pattern), стремясь уменьшить fill-in в факторизации. Пусть P — матрица перестановки, соответствующая p . Тогда

$$M_p := P M P^\top.$$

Факторизацию выполняют на M_p :

$$M_p = LL^\top \quad (\text{SPD, Cholesky}), \quad M_p = LDL^\top \quad (\text{симметрично-неопределённая, LDL}^\top).$$

Решение $Mx = b$ делается через переставленную систему:

$$M_p x_p = b_p, \quad b_p := Pb, \quad x := P^\top x_p.$$

Во многих библиотеках перестановка скрыта внутри объекта факторизации, но студент должен понимать, что **перестановка всегда применяется и к матрице, и к правой части**, а затем решение “распереставляется” обратно.

А.3 Что значит “одинаковый pattern” и как его зафиксировать

Разделение symbolic/numeric и операции update обычно требуют, чтобы все матрицы M_k или K_k имели **одинаковый шаблон ненулей**.

Практически это означает:

- в структуре матрицы должны присутствовать **все элементы, которые когда-либо будут ненулевыми**;
- на итерациях рекомендуется обновлять **только массив значений** (например, `data`) при неизменных `indices/indptr` в CSR/CSC;
- нежелательно каждый раз заново собирать матрицу через произведения `sparse-sparse`, если это может менять структуру (удаление нулей, смена порядка индексов, появление/исчезновение элементов).

Для IPM это удобно, поскольку изменяются только диагональные блоки. Поэтому pattern можно зафиксировать, собрав “базовую” матрицу с нужной структурой (включая диагонали), а затем на итерациях просто переписывать соответствующие диагональные значения.

А.4 Cholesky для SPD (нормальные уравнения)

Для нормальных уравнений

$$M_k := AX_k Z_k^{-1} A^\top + Y_k^{-1} W_k$$

матрица M_k является симметричной и (при $x, w, y, z > 0$) обычно положительно определённой. Тогда корректный выбор — **разреженная факторизация Холецкого**:

$$M_{p,k} = L_k L_k^\top, \quad M_{p,k} := P M_k P^\top.$$

Рекомендованный путь в Python: CHOLMOD через scikit-sparse. Концептуально:

1. Один раз построить M_0 с правильным pattern (например, в первой итерации IPM) и выполнить **analysis** с AMD: получить объект-фактор F (он фиксирует перестановку и структуру).
2. На каждой итерации формировать M_k с **тем же pattern** и вызывать **numeric factorization in-place** на уже проанализированном факторе.
3. Для каждого RHS вызывать solve на факторе.

Mehrotra: два RHS при одной факторизации. В методе Мехротры внутри одной итерации матрица M_k фиксирована, а RHS меняется (предиктор/корректор), поэтому делается:

1. одна numeric factorization;
2. два solve для разных RHS.

Это даёт заметное ускорение и должно быть реализовано в варианте 2.

А.5 Что можно сделать в “чистом SciPy” (без сторонних пакетов)

SciPy предоставляет разреженные прямые решатели (например, SuperLU) и удобную функцию `factorized(A)`, которая возвращает callable `solve(rhs)` для многократного решения *одной и той же* матрицы A с разными RHS. Это полезно для Мехротры (две RHS на итерацию), если вы фиксируете матрицу в пределах итерации.

Однако SciPy обычно не даёт учебно-простого интерфейса уровня:

“symbolic analysis один раз + numeric factorization много раз для матриц с тем же pattern”

поэтому переиспользование именно *symbolic* стадии между итерациями в обязательной части не требуется. Если студент реализует такой режим через сторонние библиотеки, это оценивается выше.

А.6 Редуцированная симметричная форма: AMD + (Symbolic/Numeric) sparse LDL[⊤] в Python

(1) **Какая система решается.** Используется редуцированная симметричная система на $(\Delta x, \Delta y)$:

$$K_k \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = r_k, \quad K_k := \begin{bmatrix} H_k & A^\top \\ A & -Q_k \end{bmatrix}, \quad r_k := \begin{bmatrix} \sigma + X^{-1}r_{xz} \\ \rho - Y^{-1}r_{yw} \end{bmatrix},$$

где

$$H_k := X_k^{-1}Z_k \text{ (диагональная и } H_k \succ 0), \quad Q_k := Y_k^{-1}W_k \text{ (диагональная и } Q_k \succ 0).$$

Матрица K_k симметрична и **симметрично-неопределённа**, но при $x, w, y, z > 0$ она является **quasi-definite** (верхний блок SPD, нижний блок *отрицательно* SPD). Для таких матриц существует устойчивая LDL[⊤] факторизация без pivoting, а pattern не меняется между итерациями, так как A фиксирована, а H_k, Q_k меняются только по диагонали.

(2) **Почему тут важны AMD и разделение Symbolic/Numeric.**

- AMD-переупорядочение выбирается **по pattern** матрицы K_k и может быть вычислено один раз.
- Symbolic этап (структура fill-in, дерево исключения, распределение памяти) выполняется один раз.
- Numeric этап (значения L, D) повторяется на каждой итерации, поскольку меняются диагонали H_k, Q_k .

(3) **Практический выбор библиотек в Python.** Для редуцированной формы удобна связка:

- **AMD:** `sksparse.amd` (или эквивалентный модуль AMD).
- **LDL[⊤] quasi-definite + update:** пакет `qdlldl`.

Идея: AMD фиксирует перестановку по pattern; `qdlldl` позволяет выполнять numeric update при неизменном pattern.

(4) **Алгоритм:** один раз (AMD + symbolic), затем на каждой итерации (numeric + solve). **Шаг А:** подготовка pattern (один раз).

1. Построить K_0 с *гарантированным* присутствием диагонали (важно для фиксированного pattern). Например, взять $H_0 = I_n$, $Q_0 = I_m$ и собрать $K_0 = \begin{bmatrix} H_0 & A^\top \\ A & -Q_0 \end{bmatrix}$.
2. Привести K_0 к CSC/CSR, выполнить `sum_duplicates()` и `sort_indices()`.
3. Вычислить AMD-перестановку p по pattern K_0 и зафиксировать её на всём ходе IPM.
4. Построить переставленную матрицу $K_{0,p} := K_0(p, p)$.
5. Инициализировать $\text{LDL}^\top$ факторизацию на $K_{0,p}$ (создать объект фактора; этот шаг включает symbolic анализ).

Шаг В: на каждой итерации IPM.

1. Вычислить новые диагонали $H_k = X_k^{-1}Z_k$ и $Q_k = Y_k^{-1}W_k$.
2. Собрать K_k с **тем же pattern**, что и у K_0 : $K_k = \begin{bmatrix} H_k & A^\top \\ A & -Q_k \end{bmatrix}$.
3. Применить ту же перестановку:

$$K_{k,p} := K_k(p, p), \quad r_{k,p} := r_k(p).$$

4. Выполнить **numeric update** фактора на $K_{k,p}$ (без повторного symbolic этапа).
5. Решить $K_{k,p}u_p = r_{k,p}$ и восстановить $u = P^\top u_p$, где $u = [\Delta x; \Delta y]$.
6. Восстановить оставшиеся направления:

$$\Delta z = A^\top \Delta y - \sigma, \quad \Delta w = \rho - A \Delta x.$$

(5) **Инженерная деталь:** как гарантировать “**тот же pattern**”. Update-факторизации требует одинакового pattern. Поэтому рекомендуется:

- собрать K_0 так, чтобы **все диагональные элементы** присутствовали как ненули;
- хранить матрицу в CSR/CSC и на итерациях переписывать только **data**;
- заранее вычислить позиции диагональных элементов в **data** для быстрого обновления.

(6) **Скелет кода Python (схема AMD + перестановка + update).** Ниже приведён *схематический* код (без построения RHS), показывающий места AMD, перестановки и numeric update:

```
import numpy as np
import scipy.sparse as sp

from sksparse.amd import amd
```

```

import qlddl

# A: m x n sparse matrix
# Build K0 with guaranteed diagonal pattern
H0 = sp.eye(n, format="csc")
Q0 = sp.eye(m, format="csc")
K0 = sp.bmat([[H0, A.T], [A, -Q0]], format="csc")
K0.sum_duplicates(); K0.sort_indices()

# AMD ordering (symmetric)
p = amd(K0) # permutation vector
K0p = K0[p, :][:, p].tocsc()

# Symbolic + numeric (initial) factorization
F = qlddl.Solver(K0p)

# Use CSR for fast in-place data updates
Kp = K0p.tocsr()
Kp.sort_indices()

# Precompute diagonal positions in CSR data
diag_pos = []
for i in range(Kp.shape[0]):
    a, b = Kp.indptr[i], Kp.indptr[i+1]
    cols = Kp.indices[a:b]
    j = np.searchsorted(cols, i)
    diag_pos.append(a + j)

for k in range(max_iter):
    # compute diag(H_k) and diag(Q_k), and build rhs r (size n+m)
    # IMPORTANT: you must permute diags consistently with p

    # Update diagonal entries in permuted matrix
    for i in range(n):
        Kp.data[diag_pos[i]] = Hk_diag_perm[i]
    for i in range(m):
        Kp.data[diag_pos[n+i]] = -Qk_diag_perm[i]

    # Numeric update on fixed pattern
    F.update(Kp)

    # Solve with permuted rhs
    rp = r[p]
    up = F.solve(rp)

    # Unpermute solution
    u = np.empty_like(up)
    u[p] = up

```

```

dx = u[:n]
dy = u[n:]

# recover dz, dw
dz = A.T @ dy - sigma
dw = rho - A @ dx

```

(7) Две RHS в Mehrotra (предиктор/корректор). В одной итерации Мехротры матрица K_k фиксирована, меняются только RHS. После numeric update (или factorization) выполняются два solve:

$$u_p^{\text{aff}} = K_{k,p}^{-1} r_{k,p}^{\text{aff}}, \quad u_p^{\text{tot}} = K_{k,p}^{-1} r_{k,p}^{\text{tot}}.$$

Факторизация (и перестановка) одна, solve — два раза.

(8) Численная устойчивость. Если факторизация становится неустойчивой (слишком малые диагональные элементы, рост ошибок), часто применяют регуляризацию диагональных блоков:

$$H_k \leftarrow H_k + \beta_x I, \quad Q_k \leftarrow Q_k + \beta_y I, \quad \beta_x, \beta_y > 0 \text{ малые.}$$

В учебной версии достаточно β порядка $10^{-10} \dots 10^{-8}$, с обязательным описанием в README.

A.7 Рекомендации по оформлению в README (что считается “сильной” реализацией)

Чтобы было понятно, что именно оптимизировано, в README рекомендуется указать:

- какую систему вы решаете: (11) (SPD) или редуцированную симметричную (quasi-definite);
- какой reordering используется (AMD/или другой) и где он вычисляется (один раз);
- что именно переиспользуется: перестановка, symbolic анализ, память под факторы;
- как обеспечено сохранение pattern (обновление только data);
- для Мехротры: что факторизация выполняется один раз на итерацию, а solve делается два раза (две RHS).